Lab 09 – Worksheet

Name:Abdullah Rasool Arsalan Rashid Wasay Ali	ID:ar11014 Ar12008 wa11538	Section:T2

Note: Assumptions and logics should be explained separately in tasks after the task results.

Task 1

Truth Table
a) Main Control Unit
b) ALU Control Unit

Main Control Unit — opcode decoding

Opcode	Type	Reg Write	ALU Src	Mem Read	Mem Write	Memto Reg	Branch	ALUOp
0110011	R-type	1	0	0	0	0	0	10
0010011	ADDI	1	1	0	0	0	0	00
0000011	Load	1	1	1	0	1	0	00
0100011	Store	0	1	0	1	X	0	00
1100011	BEQ	0	0	0	0	X	1	01

ALU Control Unit — operation decoding

ALUOp	funct3	funct7	Instruction / purpose	ALUControl
00	XXX	XXXXXXX	ADDI / Load / Store → ADD	0000
01	000	XXXXXXX	BEQ comparison	0001
10	000	0000000	ADD	0000
10	000	0100000	SUB	0001
10	001	0000000	SLL	0101
10	101	0000000	SRL	0110
10	111	0000000	AND	0010
10	110	0000000	OR	0011
10	100	0000000	XOR	0100

Comments

The 7-bit opcode sets RegWrite, ALUSrc, MemRead, MemWrite, MemtoReg, Branch and ALUOp. R-type instructions use two registers; ADDI, loads and stores use an immediate. Loads read memory, stores write memory and BEQ enables Branch. The ALU decoder uses ALUOp, funct3 and funct7, retaining Lab 6 ALUControl encodings. For LB/LH/LW and SB/SH/SW, funct3 = 000/001/010 indicates byte/halfword/word width. ALUOp 00 = ADD, 01 = BEQ and 10 = R-type. X means don't care; safe values are used in Verilog.

Originally the truth table we designed had ALUCONTROL for BEQ 0111 and for SUB 0001, but since we are following a single path RISC-V cycle, it should be the same for both. So we redesigned the table and the fpga implementation.

Task 2

*Design Module Codes for **Main Control Unit,** and **ALU Control Unit.***

MainControl.v

```
`timescale 1ns / 1ps

module MainControl(input [6:0] opcode,
 output reg RegWrite, output reg [1:0] ALUOp,
 output reg MemRead, MemWrite, ALUSrc, MemtoReg, Branch);
 always @(*) begin
  {RegWrite,ALUSrc,MemRead,MemWrite,MemtoReg,Branch,ALUOp}
     = 8'b00000011; // safe default: ALUOp=11
  case (opcode)
   7'b0110011: {RegWrite,ALUSrc,MemRead,MemWrite,
     MemtoReg,Branch,ALUOp} = 8'b10000010; // R-type
   7'b0010011: {RegWrite,ALUSrc,MemRead,MemWrite,
     MemtoReg,Branch,ALUOp} = 8'b11000000; // ADDI
   7'b0000011: {RegWrite,ALUSrc,MemRead,MemWrite,
     MemtoReg,Branch,ALUOp} = 8'b11101000; // LB/LH/LW
   7'b0100011: {RegWrite,ALUSrc,MemRead,MemWrite,
     MemtoReg,Branch,ALUOp} = 8'b01010000; // SB/SH/SW
   7'b1100011: {RegWrite,ALUSrc,MemRead,MemWrite,
     MemtoReg,Branch,ALUOp} = 8'b00000101; // BEQ
   default: ;
  endcase
 end
```

```
endmodule
```

ALUControl.v

```
`timescale 1ns / 1ps

module ALUControlUnit(input [1:0] ALUOp,
 input [2:0] funct3, input [6:0] funct7,
 output reg [3:0] ALUControl);
 always @(*) begin
  ALUControl = 4'b1111; // unsupported
  case (ALUOp)
   2'b00: ALUControl = 4'b0000; // ADDI/load/store
   2'b01: if (funct3==3'b000) ALUControl=4'b0111;
   2'b10: case ({funct7,funct3})
    {7'b0000000,3'b000}: ALUControl=4'b0000; // ADD
    {7'b0100000,3'b000}: ALUControl=4'b0001; // SUB
    {7'b0000000,3'b001}: ALUControl=4'b0101; // SLL
    {7'b0000000,3'b101}: ALUControl=4'b0110; // SRL
    {7'b0000000,3'b111}: ALUControl=4'b0010; // AND
    {7'b0000000,3'b110}: ALUControl=4'b0011; // OR
    {7'b0000000,3'b100}: ALUControl=4'b0100; // XOR
    default: ALUControl = 4'b1111;
   endcase
   default: ALUControl = 4'b1111;
  endcase
 end
endmodule
```

Testbench:

ControlPath_tb.v (full self-checking Verilog testbench is provided below)

Connects MainControl and ALUControlUnit. For each vector, sets opcode,
funct3 and funct7, waits 10 ns, and checks all main signals and ALUControl.
Tests: ADD, SUB, SLL, SRL, AND, OR, XOR, ADDI, LW, LH, LB,
SW, SH, SB, BEQ; plus invalid opcode, R-type and branch encoding.
Expected: 18 test vectors, 0 failures, "ALL TESTS PASSED".
Vivado: set ControlPath_tb as Simulation Top, Run Behavioral Simulation.

ControlPath_tb.v:

```
`timescale 1ns / 1ps

module ControlPath_tb;

    reg [6:0] opcode;
    reg [2:0] funct3;
    reg [6:0] funct7;

    wire RegWrite, MemRead, MemWrite;
    wire ALUSrc, MemtoReg, Branch;
    wire [1:0] ALUOp;
    wire [3:0] ALUControl;

    wire [7:0] mainSignals = {
        RegWrite, ALUSrc, MemRead,
        MemWrite, MemtoReg, Branch, ALUOp
    };

    localparam [7:0] R_TYPE  = 8'b10000010;
    localparam [7:0] ADDI_T  = 8'b11000000;
    localparam [7:0] LOAD_T  = 8'b11101000;
    localparam [7:0] STORE_T = 8'b01010000;
    localparam [7:0] BEQ_T   = 8'b00000101;
    localparam [7:0] BAD_T   = 8'b00000011;

    integer tests = 0;
    integer errors = 0;

    MainControl main_unit (
        .opcode(opcode),
        .RegWrite(RegWrite),
        .ALUOp(ALUOp),
        .MemRead(MemRead),
        .MemWrite(MemWrite),
        .ALUSrc(ALUSrc),
        .MemtoReg(MemtoReg),
        .Branch(Branch)
    );

    ALUControlUnit alu_unit (
        .ALUOp(ALUOp),
        .funct3(funct3),
```

```
    .funct7(funct7),
    .ALUControl(ALUControl)
);

task check_instruction;
    input [8*16-1:0] name;
    input [6:0] test_opcode;
    input [2:0] test_funct3;
    input [6:0] test_funct7;
    input [7:0] expected_main;
    input [3:0] expected_alu;

    begin
        opcode = test_opcode;
        funct3 = test_funct3;
        funct7 = test_funct7;

        #10;

        tests = tests + 1;

        if ((mainSignals !== expected_main) ||
            (ALUControl !== expected_alu)) begin

            errors = errors + 1;

            $display("FAIL %-16s main=%b expected=%b ALU=%b expected=%b",
                name, mainSignals, expected_main,
                ALUControl, expected_alu);

        end else begin

            $display("PASS %-16s main=%b ALU=%b",
                name, mainSignals, ALUControl);

        end
    end
endtask

initial begin

    opcode = 7'd0;
    funct3 = 3'd0;
```

```
funct7 = 7'd0;

// R-Type Instructions

check_instruction("ADD",
   7'b0110011, 3'b000, 7'b0000000,
   R_TYPE, 4'b0000);

check_instruction("SUB",
   7'b0110011, 3'b000, 7'b0100000,
   R_TYPE, 4'b0001);

check_instruction("SLL",
   7'b0110011, 3'b001, 7'b0000000,
   R_TYPE, 4'b0101);

check_instruction("SRL",
   7'b0110011, 3'b101, 7'b0000000,
   R_TYPE, 4'b0110);

check_instruction("AND",
   7'b0110011, 3'b111, 7'b0000000,
   R_TYPE, 4'b0010);

check_instruction("OR",
   7'b0110011, 3'b110, 7'b0000000,
   R_TYPE, 4'b0011);

check_instruction("XOR",
   7'b0110011, 3'b100, 7'b0000000,
   R_TYPE, 4'b0100);

// ADDI

check_instruction("ADDI",
   7'b0010011, 3'b000, 7'b0110101,
   ADDI_T, 4'b0000);

// Load Instructions

check_instruction("LW",
   7'b0000011, 3'b010, 7'b0000000,
   LOAD_T, 4'b0000);
```

```
check_instruction("LH",
    7'b0000011, 3'b001, 7'b0000011,
    LOAD_T, 4'b0000);

check_instruction("LB",
    7'b0000011, 3'b000, 7'b1010101,
    LOAD_T, 4'b0000);

// Store Instructions

check_instruction("SW",
    7'b0100011, 3'b010, 7'b0000000,
    STORE_T, 4'b0000);

check_instruction("SH",
    7'b0100011, 3'b001, 7'b1110010,
    STORE_T, 4'b0000);

check_instruction("SB",
    7'b0100011, 3'b000, 7'b1000110,
    STORE_T, 4'b0000);

// Branch Instruction

check_instruction("BEQ",
    7'b1100011, 3'b000, 7'b0000000,
    BEQ_T, 4'b0111);

// Invalid Input Tests

check_instruction("BAD_OPCODE",
    7'b1111111, 3'b000, 7'b0000000,
    BAD_T, 4'b1111);

check_instruction("BAD_RTYPE",
    7'b0110011, 3'b000, 7'b0000001,
    R_TYPE, 4'b1111);

check_instruction("BAD_BRANCH",
    7'b1100011, 3'b001, 7'b0000000,
    BEQ_T, 4'b1111);
```

```
        $display("TOTAL: %0d tests; %0d failures",
            tests, errors);

        if (errors == 0)
            $display("ALL TESTS PASSED");
        else begin
            $display("TESTS FAILED");
            $stop;
        end

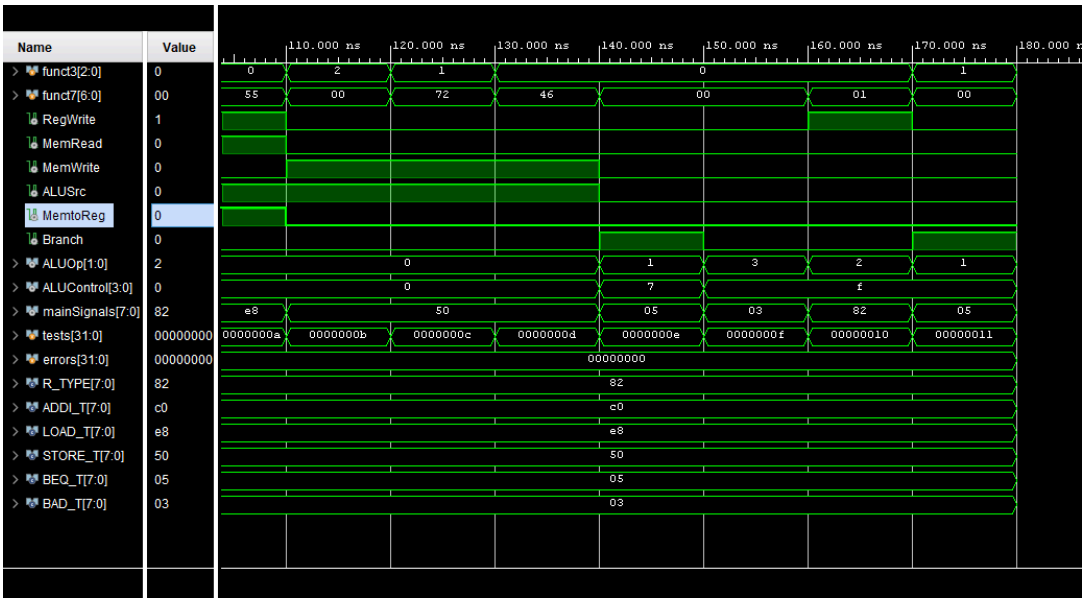
        $finish;

    end

endmodule
```

Waveforms:


Name
Value
funct3[2:0]
funct7[6:0]
RegWrite
MemRead
MemWrite
ALUSrc
MemtoReg
Branch
ALUOp[1:0]
ALUControl[3:0]
mainSignals[7:0]
tests[31:0]
errors[31:0]
R_TYPE[7:0]
ADDI_T[7:0]
LOAD_T[7:0]
STORE_T[7:0]
BEQ_T[7:0]
BAD_T[7:0]
0.000 ns
10.000 ns
20.000 ns
30.000 ns
40.000 ns
50.000 ns
60.000 ns
70.000 ns
80.000 ns
90.000 ns
100.000 ns
110.000 ns
120.000 ns
130.000 ns
140.000 ns
150.000 ns



Waveform signals: opcode, funct3, funct7, RegWrite, ALUOp, MemRead, MemWrite, ALUSrc, MemtoReg, Branch, ALUControl. Expected checkpoints: ADD=0000; SUB=0001; SLL=0101; SRL=0110; AND=0010; OR=0011; XOR=0100; BEQ=0111. Waveform succeeds as expected.

Task 3

Design module codes of ***top module:***

```
`timescale 1ns / 1ps
module ControlPath_FPGA_Top(
 input wire clk,
 input wire pbin,
 input wire [15:0] physical_sw,
 output wire [15:0] physical_leds
);
 wire btn_clean;
 wire [31:0] switch_data;
 wire [31:0] led_write_data;
 wire [15:0] led_out;
 reg [6:0] opcode_reg = 7'd0;
 reg [2:0] funct3_reg = 3'd0;
 reg [6:0] funct7_reg = 7'd0;
 reg [1:0] state = 2'd0;
 // WAIT_PRESS = 0; WAIT_RELEASE = 1
```

```
localparam WAIT_PRESS=2'd0, WAIT_RELEASE=2'd1;
wire RegWrite, MemRead, MemWrite, ALUSrc, MemtoReg, Branch;
wire [1:0] ALUOp;
wire [3:0] ALUControl;
wire invalid;

debouncer button_filter(.clk(clk),.pbin(pbin),.pbout(btn_clean));
// Names match the Lab 5 supplied interface modules.
leds switch_reader(
  .clk(clk),.rst(1'b0),.btns(16'd0),.writeData(32'd0),
  .writeEnable(1'b0),.readEnable(1'b1),.memAddress(30'd0),
  .switches(physical_sw),.readData(switch_data)
);

always @(posedge clk) begin
  if (switch_data[13]) begin
    opcode_reg <= 7'b0110011;
    funct3_reg <= 3'b000;
    funct7_reg <= 7'b0000000;
    state <= WAIT_PRESS;
  end else begin
    case(state)
      WAIT_PRESS: if (btn_clean) begin
        opcode_reg <= switch_data[6:0];
        funct3_reg <= switch_data[9:7];
        funct7_reg <= {1'b0, switch_data[10], 5'b00000};
        state <= WAIT_RELEASE;
      end
      WAIT_RELEASE: if (!btn_clean) state <= WAIT_PRESS;
      default: state <= WAIT_PRESS;
    endcase
  end
end

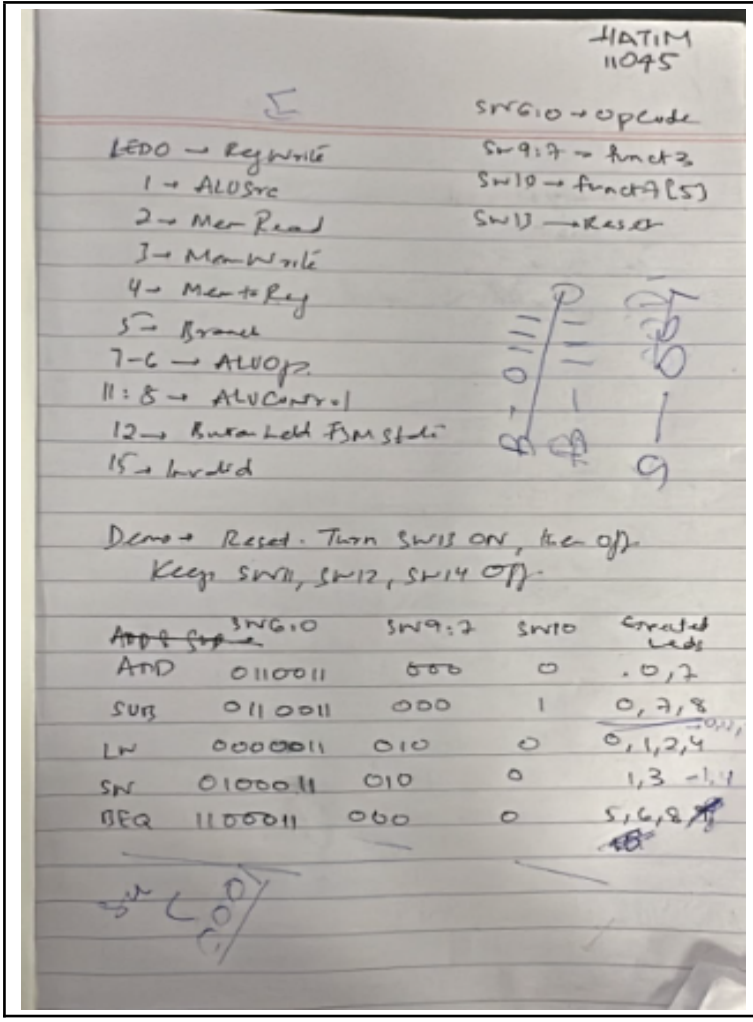
MainControl main_control(
  .opcode(opcode_reg),.RegWrite(RegWrite),.ALUOp(ALUOp),
  .MemRead(MemRead),.MemWrite(MemWrite),.ALUSrc(ALUSrc),
  .MemtoReg(MemtoReg),.Branch(Branch)
);
ALUControlUnit alu_control(
  .ALUOp(ALUOp),.funct3(funct3_reg),.funct7(funct7_reg),
  .ALUControl(ALUControl)
);
assign invalid = (ALUOp == 2'b11) || (ALUControl == 4'b1111);
assign led_write_data = {16'd0,invalid,2'b00,
```

```
      (state == WAIT_RELEASE),ALUControl,ALUOp,
      Branch,MemtoReg,MemWrite,MemRead,ALUSrc,RegWrite};
 switches led_writer(
  .clk(clk),.rst(1'b0),.writeData(led_write_data),
  .writeEnable(1'b1),.readEnable(1'b0),.memAddress(30'd0),
  .readData(),.leds(led_out)
 );
 assign physical_leds = led_out;
endmodule
```

FSM:

FSM STATE TABLE:

Current State	Condition	Action	Next State
WAIT_PRESS	BTN_C = 0	Wait	WAIT_PRESS
WAIT_PRESS	BTN_C = 1	Get opcode, func3, func7	WAIT_RELEASE
WAIT_RELEASE	BTN_C = 1	Hold captured instruction	WAIT_RELESE
WAIT_RELEASE	BTN_C = 0	Ready for next instruction	WAIT_PRESS
Any State	Reset = 1	Load default instruction (ADD)	WAIT_PRESS



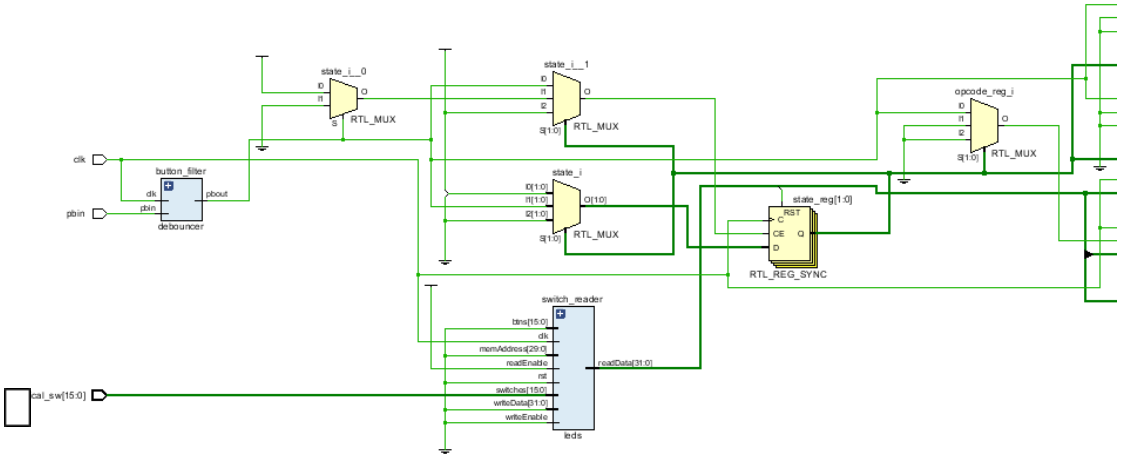
Constraints File:

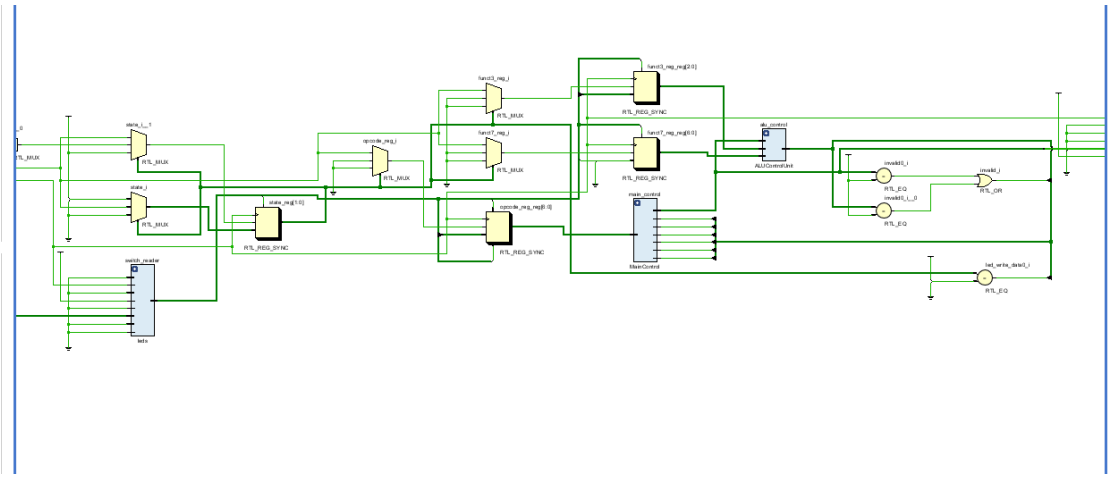
```
set_property PACKAGE_PIN W5 [get_ports clk]
set_property IOSTANDARD LVCMOS33 [get_ports clk]
create_clock -period 10.000 -name sys_clk [get_ports clk]
set_property PACKAGE_PIN U18 [get_ports pbin]
set_property IOSTANDARD LVCMOS33 [get_ports pbin]
set_property PACKAGE_PIN V17 [get_ports {physical_sw[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_sw[0]}]
set_property PACKAGE_PIN V16 [get_ports {physical_sw[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_sw[1]}]
set_property PACKAGE_PIN W16 [get_ports {physical_sw[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_sw[2]}]
```

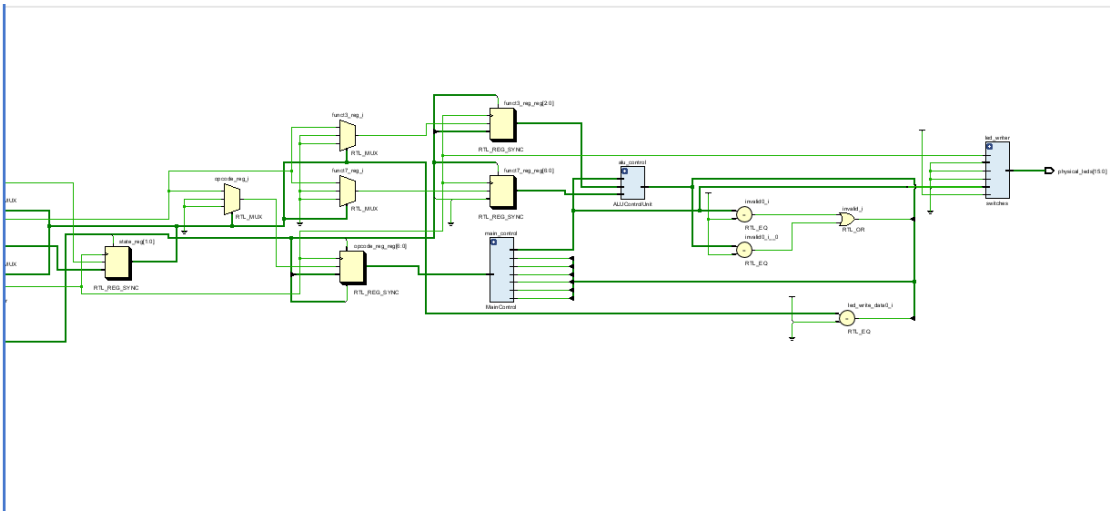
```
set_property PACKAGE_PIN W17 [get_ports {physical_sw[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_sw[3]}]
set_property PACKAGE_PIN W15 [get_ports {physical_sw[4]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_sw[4]}]
set_property PACKAGE_PIN V15 [get_ports {physical_sw[5]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_sw[5]}]
set_property PACKAGE_PIN W14 [get_ports {physical_sw[6]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_sw[6]}]
set_property PACKAGE_PIN W13 [get_ports {physical_sw[7]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_sw[7]}]
set_property PACKAGE_PIN V2 [get_ports {physical_sw[8]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_sw[8]}]
set_property PACKAGE_PIN T3 [get_ports {physical_sw[9]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_sw[9]}]
set_property PACKAGE_PIN T2 [get_ports {physical_sw[10]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_sw[10]}]
set_property PACKAGE_PIN R3 [get_ports {physical_sw[11]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_sw[11]}]
set_property PACKAGE_PIN W2 [get_ports {physical_sw[12]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_sw[12]}]
set_property PACKAGE_PIN U1 [get_ports {physical_sw[13]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_sw[13]}]
set_property PACKAGE_PIN T1 [get_ports {physical_sw[14]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_sw[14]}]
set_property PACKAGE_PIN R2 [get_ports {physical_sw[15]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_sw[15]}]
set_property PACKAGE_PIN U16 [get_ports {physical_leds[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_leds[0]}]
set_property PACKAGE_PIN E19 [get_ports {physical_leds[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_leds[1]}]
set_property PACKAGE_PIN U19 [get_ports {physical_leds[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_leds[2]}]
set_property PACKAGE_PIN V19 [get_ports {physical_leds[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_leds[3]}]
set_property PACKAGE_PIN W18 [get_ports {physical_leds[4]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_leds[4]}]
set_property PACKAGE_PIN U15 [get_ports {physical_leds[5]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_leds[5]}]
set_property PACKAGE_PIN U14 [get_ports {physical_leds[6]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_leds[6]}]
set_property PACKAGE_PIN V14 [get_ports {physical_leds[7]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_leds[7]}]
set_property PACKAGE_PIN V13 [get_ports {physical_leds[8]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_leds[8]}]
set_property PACKAGE_PIN V3 [get_ports {physical_leds[9]}]
```

```
set_property IOSTANDARD LVCMOS33 [get_ports {physical_leds[9]}]
set_property PACKAGE_PIN W3 [get_ports {physical_leds[10]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_leds[10]}]
set_property PACKAGE_PIN U3 [get_ports {physical_leds[11]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_leds[11]}]
set_property PACKAGE_PIN P3 [get_ports {physical_leds[12]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_leds[12]}]
set_property PACKAGE_PIN N3 [get_ports {physical_leds[13]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_leds[13]}]
set_property PACKAGE_PIN P1 [get_ports {physical_leds[14]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_leds[14]}]
set_property PACKAGE_PIN L1 [get_ports {physical_leds[15]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_leds[15]}]
```

RTL Schematic:







Lab 9: Design and FPGA Implementation of the Control Path for a Single-Cycle RISC-V Processor

Points Distribution

Task No.	LR2 Code	LR 5 Results	LR11 Design	AR7
Task 1	-	-	/20	/20
Task 2	/20	/15		
Task 3	/10	/15		
Total Points	/80			/20
CLO Mapped	CLO 1			

For description of different levels of the mapped rubrics, please refer the provided Lab Evaluation Assessment Rubrics and Affective Domain Assessment Rubrics.

#	Assessment Elements	Level 1: Unsatisfactory Points 0-1	Level 2: Developing Points 2	Level 3:Good Points 3	Level 4:Exemplary Points 4
LR2	Program/Code / Simulation Model/ Network Model	Program/code/simulation model/network model does not implement the required functionality and has several errors. The student is not able to utilize even the basic tools of the software.	Program/code/simulation model/network model has some errors and does not produce completely accurate results. Student has limited command on the basic tools of the software.	Program/code/simulation model/network model gives correct output but not efficiently implemented or implemented by computationally complex routine.	Program/code/simulation /network model is efficiently implemented and gives correct output. Student has full command on the basic tools of the software.
LR5	Results & Plots	Figures/ graphs / tables are not developed or are poorly constructed with erroneous results. Titles, captions, units are not mentioned. Data is presented in an obscure manner.	Figures, graphs and tables are drawn but contain errors. Titles, captions, units are not accurate. Data presentation is not too clear.	All figures, graphs, tables are correctly drawn but contain minor errors or some of the details are missing.	Figures / graphs / tables are correctly drawn and appropriate titles/captions and proper units are mentioned. Data presentation is systematic.
AR9	Report	All the in-lab tasks are not included in report and / or the report is submitted too late.	Most of the tasks are included in report but are not well explained. All the necessary figures / plots are not included.	Good summary of most the in-lab tasks is included in report. The work is supported by figures and plots with explanations.	Detailed summary of the in-lab tasks is provided. All tasks are included and explained well. Data is presented clearly including all

			Report is submitted after due date.	The report is submitted timely.	the necessary figures, plots and tables.